CogniFlow - Demo Video Script

Every word you say, in order. One narrator. Target length 4 minutes 30.

How to use this document

The blue panels are the **only** things you read aloud. Everything outside them is instruction to you. Each PART is a self-contained take - if you are recording in segments, stop between PARTs and start the next one fresh. The companion guide explains what every term means and tells you when to read which PART.

PART	What it covers	Clock	Length
0	Setup - nothing spoken	before	-
1	The problem	0:00 - 0:30	30s
2	What is and is not scripted	0:30 - 0:50	20s
3	Start the run, first attempt fails	0:50 - 1:35	45s
4	THE MOMENT - it changes its own goal	1:35 - 2:25	50s
4b	The guard overruling the model (if it appears)	within 4	10s
5	Infrastructure failure, mastery untouched	2:25 - 2:55	30s
6	The return to recursion	2:55 - 3:20	25s
7	It checks its own claims	3:20 - 3:45	25s
8	Does it actually help - the numbers	3:45 - 4:20	35s
9	Close	4:20 - 4:35	15s
10	Live interrupt add-on (only if under time)	extra	20s

PART 0 - Before you press record

Nothing here is spoken. Do all of it first.

Run these and confirm they pass:

```
cd D:\Downloads\BloodCoded
python run.py check      # one real call per AI provider
python run.py live       # the full demo against real models
```

Run `python run.py live` **three times**. Two things must match every time, and they are the two the demo actually claims:

Must be identical	What to look for
The learning path	Learning path : recursion -> functions -> recursion
The self-checks	all 7 [PASS] lines, and no [FAIL]

Other things will differ between runs, and none of them are faults. Measured over three consecutive runs on 5 September:

- **The exercise titles change.** The AI writes a new exercise every run.
- **The number of steps can change.** The scripted student submits fixed code, and a differently worded exercise may not accept it. In one of the three runs a functions answer was graded wrong, and the agent responded by explaining differently and lowering the difficulty - an extra loop, and arguably a better demo than the shorter runs.
- **The provider changes.** Groq until its per-minute ceiling, then Google.
- **The duration changes.** 42 to 65 seconds observed.

That variation is the *student* being scripted while the *tutor* is not - which is exactly what we are claiming. **Only stop and tell Sohan if the learning path itself changes, or if any check says FAIL.**

Screen and audio

- Terminal about 110 characters wide, font large enough to read on a phone.
- Close everything else: notifications, browser tabs, chat apps.
- Record ten seconds of yourself and play it back before a real take.
- Have this script on a second screen or printed - not on the screen you record.

You are allowed to record in segments

You are doing two jobs at once - driving the terminal and narrating. Recording each PART as its own take and joining them afterwards is completely legitimate and much easier than one continuous take. Nothing is fabricated: the output on screen is still a real run. The rulebook forbids faking **results**, and you are not faking anything.

The easier way to narrate a long run

A live run makes real API calls, so there are pauses while it thinks. Two options, both honest: **(a)** narrate as it scrolls, mentioning the pauses - "that is a live API call" - or **(b)** let the whole run finish, then scroll back up through the finished output and narrate over it. Option (b) is far easier alone and you control the pace completely, and it is still the genuine output of a real run. Pick one in rehearsal and stick to it.

PART 1 - The problem

Clock	On screen
0:00 - 0:30	The seeded student model - the skills list with their mastery numbers

DO: Have the starting model visible. Point at 'functions 0.55' and 'recursion 0.35' as you say them.

READ ALOUD:

> A student is learning Python. They fail a recursion exercise. Then they fail another one.
>
> Every AI tutor available today does the same thing here. It gives them an easier recursion problem. Maybe it adds a hint. Maybe it explains recursion again, more slowly.
>
> But the student's real problem usually is not recursion at all. It is that they do not understand what *return* does when one function calls another. Give them an easier recursion problem and they will fail that too, for exactly the same invisible reason.
>
> CogniFlow is built to notice that, and to go back a step.

Delivery: this is the whole pitch. Slow down. Do not rush to the terminal.

Point at the two numbers as you name them - functions 0.55, recursion 0.35.

PART 2 - What is and is not scripted

Clock	On screen
0:30 - 0:50	Still the seeded model, or the command about to be run

DO: None - just speak.

READ ALOUD:

> One thing before I run it. The *student* is scripted - what they submit, and when. Nothing the tutor does is scripted.
>
> Every decision you are about to see is computed live, from a prerequisite graph and from Bayesian estimates of what this student knows.
>
> And at the end, the demo checks its own claims - so you do not have to take my word for any of it.

Why this exists: it answers 'is this hardcoded?' before a judge has to ask.

PART 3 - Start the run - the first attempt fails

Clock	On screen
0:50 - 1:35	The live event stream, from [diagnose] down to the first [adapt]

DO: Run 'python run.py live' now. Let the first block scroll.

READ ALOUD:

> Real diagnosis, real retrieval with citations back to the source page, real code execution in a sandbox.
>
> It has worked out that recursion is the weakest skill, and that recursion depends on functions. It searched our own course notes and pulled the recursion section - you can see the file and heading it came from.
>
> The student's code raised an error. Mastery drops from 0.35 to 0.20, and the agent retries a variation. So far, this is a normal tutor.

Point at, in order: [diagnose], then [retrieve] and its evidence line, then [execute] and [mastery].

If it pauses while a model is called: 'that is a live API call - this is not a recording.'

PART 4 - THE MOMENT - it changes its own goal

Clock	On screen
1:35 - 2:25	[misconception] ... implicates=functions, then [adapt] REVISIT_PREREQUISITE, then [prereq_redirect] recursion -> functions

DO: STOP SCROLLING. Leave these lines on screen for the whole of this PART.

READ ALOUD:

> There. Second failure. And the agent did *not* generate an easier recursion problem.
>
> Look at what it did instead. It read the actual error - a NoneType arithmetic failure - and diagnosed the cause: the recursive call is computed, but never returned. That is a *return* problem. Which is a functions problem wearing a recursion costume.
>
> So it walked the prerequisite graph, found that functions was the weakest unmastered thing recursion depends on, and reassigned its own teaching objective.
>
> The teaching mode changed too - to code tracing - so it is not repeating the approach that already failed. And retrieval followed it: it is now pulling *The call stack* from the functions chapter instead.

This is the entire project. Give it room. A two-second silence here is good.

The most common way to ruin this video is to scroll past this moment.

PART 4b - The guard overruling the model (optional)

Clock	On screen
inside PART 4	A [guard_override] line: proposed=... final=... violated=[...]

DO: Only read this if a [guard_override] line actually appeared. It usually does on a live run and never on an offline one. If it is not there, skip this PART.

READ ALOUD:

> And here is something worth pausing on. The model proposed jumping to the prerequisite after only one failure. The guard rejected it - one failure is not enough evidence - and forced a retry instead.
>
> That is our architecture in one line: the model proposes, and deterministic code disposes. The model cannot corrupt a learning path, and every time it is overruled, we log it.

This beat is a gift when it appears - live proof of the guard working.

It appeared in all three of our test runs, but do not invent it if it is not on screen.

PART 5 - Infrastructure failure

Clock	On screen
2:25 - 2:55	[execute] status=sandbox_error, then [recover] ... mastery_untouched=True

DO: Point directly at the mastery number and hold there while you speak.

READ ALOUD:

> Now we inject a sandbox failure in the middle of the session. Watch the mastery number.
>
> It does not move.
>
> Student outcomes and system faults are separate types in the code, and mastery can only be reached from the first one. If *our* infrastructure breaks, the student does not pay for it. That is enforced by the type system - not by a conditional that somebody has to remember to write.

Let 'It does not move' sit on its own. Short sentence, then a beat.

PART 6 - The return

Clock	On screen
2:55 - 3:20	[mastery] functions climbing, then [prereq_return] returning_to=recursion, then recursion climbing to 0.877

DO: Scroll to show the learning path summary line at the end.

READ ALOUD:

> Functions is mastered. So the agent pops its return stack and goes back to what the student originally came for - which it never forgot.
>
> Recursion: 0.35 to 0.88.
>
> Recursion, to functions, and back to recursion. That path was computed, not scripted.

PART 7 - It checks its own claims

Clock	On screen
3:20 - 3:45	The seven [PASS] lines under SELF-VERIFICATION

DO: Make sure all seven PASS lines are visible at once.

READ ALOUD:

> The demo verifies itself. Seven checks, and they assert against the event stream and the database - not against printed text.
>
> The same path is asserted in our test suite. A hardcoded narration could not pass these.

If you name any, read two: 'infrastructure fault did not move mastery' and 'returned to the original objective'.

PART 8 - Does it actually help

Clock	On screen
3:45 - 4:20	The ablation table from 'python run.py ablation'

DO: Run 'python run.py ablation' - or have its output ready to scroll to.

READ ALOUD:

> Finally: does any of this actually help? Eighty simulated students, with prerequisite gaps deliberately planted, so detection can be scored exactly.
>
> Prerequisite-aware adaptation reaches mastery in nineteen percent fewer attempts, and finds the planted gap fifty-seven percent of the time. The arm without a skill graph finds it zero percent of the time - it cannot, by construction.
>
> And the honest cost: it still redirects seven and a half percent of students who had no gap at all. That was twenty-five percent until we required evidence before allowing a detour. Diagnosis is not free, and we report what it costs.

Saying the 7.5% out loud is deliberate. Volunteering a weakness makes every other number more believable.

PART 9 - Close

Clock	On screen
4:20 - 4:35	Anything - the final summary or the learning path line

DO: None.

READ ALOUD:

> Three of eleven nodes call a model. Diagnosis, routing and mastery are deterministic and unit-tested, because arithmetic is already correct and free.
>
> The model proposes. The guard disposes. That is CogniFlow.

Stop talking after 'That is CogniFlow.' Do not add anything.

PART 10 - Live interrupt add-on (optional)

Clock	On screen
extra 20s	The web UI - 'Be the student' mode, suspended at await_student

DO: Only if comfortably under 5 minutes. Run 'python run.py ui', choose 'Be the student', click Start session.

READ ALOUD:

> One last thing. The graph is suspended right now, checkpointed to disk. It is not looping and waiting - it has genuinely stopped, and a completely separate process can pick this session up and continue it.
>
> When I submit an answer, it resumes from that checkpoint.

Skip entirely if near five minutes. Going over is worse than leaving it out.

If you do include it, submitting wrong code shows the tutor's feedback.

If something goes wrong while recording

The rule

Say what happened, plainly, and carry on. Do not talk over it and do not pretend it did not happen. A failure you recover from on camera demonstrates the recovery path we are claiming - better evidence than a run where nothing went wrong. And if a take is genuinely spoiled, just record that PART again.

What you see	What it means	Say this
degraded=True on a line	An AI provider failed, so a built-in template was used instead	"The provider is rate-limited there, so it fell back to a deterministic template. The tutoring decisions are unaffected - the model does not make those."
provider=google:...	The first provider hit its limit and it failed over automatically	"That is the provider chain failing over live." A feature - mention it.
A long pause	Waiting for a real API response	"That is a live API call - this is not a recording."
An error you do not recognise	Unknown	"Something went wrong there - let me re-run it." Then run 'python run.py verify', which needs no internet.
The learning path came out different	The one thing that should never vary	Stop recording. Tell Sohan before continuing.

Two things never to say

- Never claim something the screen is not showing. If you cannot see it, do not say it.
- Never say the system does something it does not. If asked about logins, dashboards or a diagnostic quiz - those do not exist yet, and saying so plainly is the right answer.